6 Dynamic authorization models

This chapter covers

- How authorization evolved from simple role-based controls to dynamic, context-aware authorization
- When and why to use relationship-based access control (ReBAC) for collaborative systems
- How attribute-based access control (ABAC) enables fine-grained, real-time decisions
- How policy-based access control (PBAC) unifies roles, attributes, and relationships
- The tradeoffs and strengths of each model, and how to choose the right combination for your system

At first, ACME Corp's switch to Role-Based Access Control (RBAC) seemed like a major breakthrough. After years of scattered Access Control Lists and inconsistent permissions, establishing a clear set of roles—customer support, sales manager, engineer, senior engineer—brought much-needed order to their internal systems. Onboarding got faster. Audits got cleaner. Developers could finally reason about access without digging through nested conditionals. But that clarity didn't last.

As the company expanded into global markets and adopted more collaborative, multi-tenant services, cracks began to emerge. Roles multiplied, permissions drifted, and employees—especially those who wore multiple hats—began to complain.

Alice, now a senior engineer, spent increasing amounts of time on cross-functional project work. She constantly bumped into permissions she didn't have—even though, on paper, she was clearly the right person for the job. The system was secure, but it got in the way of Alice doing her job. "I don't need access to everything," she told the identity team, "just the stuff I've been assigned to."